Sentinel — Debug Report

Scope: full repo audit (static analysis + dependency install + `pytest tests/ -v` + manual verification against the psycopg3 API). 228/230 tests pass; the 2 failures are DB-integration tests that require a live Postgres.

CRITICAL

1. Entire DB repository layer is broken — `sentinel/db/repo.py`

Every function calls `.fetchone()` / `.fetchall()` directly on the connection object:

```python
async with _conn() if conn is None else _noop(conn) as c:
    row = await c.fetchone(sql, (...))
```

`psycopg` v3's `AsyncConnection` has no `fetchone`/`fetchall` method — only `.execute()`, which returns a cursor, and the cursor is what has `fetchone`/`fetchall`. Verified directly:

```python
>>> hasattr(psycopg.AsyncConnection, 'fetchone')
False
```

Impact: `create_case`, `get_case`, `update_case_status`, `insert_evidence`, `append_audit_event`, `save_rule`, `list_rules`, `verify_audit_chain` — i.e. every DB call used by `gateway.py`, `worker.py`, `sse_worker.py` — will raise `AttributeError` the moment they hit a real Postgres connection. Never caught in CI because the only two tests that exercise this path are `@pytest.mark.integration` and just time out on connection-refused (no live Postgres in test env), so the `AttributeError` itself was never surfaced.

Fix: replace pattern with

```python
cur = await c.execute(sql, params)
row = await cur.fetchone()   # or await cur.fetchall()
```

throughout the file, or wrap connections in a helper that provides `fetchone`/`fetchall` convenience methods.

Locations (13 call sites): `repo.py` lines 73, 104, 121, 162, 199, 237, 243, 271, 305, 343, 368, 397, 416.

---

2. Ticket #18 "Confidence-Calibrated Silence" is documented as shipped but not implemented

`Sentinel_Winning_Edition_Bible.md` / `Sentinel_Master_Reference.md` describe this as a working differentiator (agents disagree / thin evidence → calm "need more info" card instead of a guess).

In code:

- `Claim.confidence` is stored on every claim but **never read** in `reconciler.py` or `pipeline.py`.
- There is no third verdict state beyond `CLEAR / REVIEW / FRAUD_LIKELY` — no "uncertain"/"need more info" branch anywhere.
- `test_silence.py` (repo root, not `tests/`) is the only artifact referencing this ticket. It is not a real pytest test — no `test_*` function, no assertions, just prints. It's also outside the path your README tells people to run (`pytest tests/ -v`), so it never executes in CI.

Ran it directly — both cases that should trigger "need more info" instead resolve silently to `CLEAR`:

```
Test 1: Thin Evidence (<2 claims)   → Verdict: CLEAR
Test 2: Low Confidence (avg < 0.3)  → Verdict: CLEAR
```

Fix: either implement confidence-gating in `reconcile()`/`run_case()` (e.g. require ≥N claims and min average confidence before emitting a definitive verdict, else return an `UNCERTAIN`/`NEEDS_INFO` verdict), or move `test_silence.py` into `tests/` with real assertions and stop describing the feature as built in the pitch docs until it exists.

BUGS / MISCONFIGURATIONS

3. Invalid/unsafe CORS config — `gateway.py`

```python
api.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
```

`allow_origins=["*"]` + `allow_credentials=True` is an invalid combination per the CORS spec. Starlette's actual behavior in this case is to echo back the request's `Origin` header instead of `*` — meaning this is effectively "allow any origin, with credentials," which is worse than a plain wildcard. **Fix:** drop `allow_credentials` if you truly want wildcard origins, or replace `["*"]` with an explicit origin allow-list.

4. Missing dependency: `aiohttp`

`slack_bolt.async_app` (imported transitively via `AsyncSlackRequestHandler` in `gateway.py`) requires `aiohttp`, which is not listed in `requirements.txt`. Following the README's own quickstart (`pip install -r requirements.txt` → `uvicorn sentinel.gateway:api`) fails immediately with:

```
ModuleNotFoundError: No module named 'aiohttp'
```

Fix: add `aiohttp` to `requirements.txt`.

5. Silent empty-string fallback for Slack secrets — `slack_app.py`

```python
app = AsyncApp(
    token=os.environ.get("SLACK_BOT_TOKEN", ""),
    signing_secret=os.environ.get("SLACK_SIGNING_SECRET", ""),
)
```

If env vars are unset, the app boots anyway with empty strings instead of failing fast. Signature verification with an empty signing secret is a security footgun, and any failure surfaces confusingly at request time instead of at startup. **Fix:** raise at import/startup if either var is missing/empty (same pattern already used correctly in `pii_gateway.py`'s `_pepper()`).

MINOR / CLEANUP

- **Unregistered pytest marker:** `@pytest.mark.integration` used in `test_gateway.py` but never declared in `pytest.ini` → `PytestUnknownMarkWarning` on every run. Add:

```ini
[pytest]
markers =
    integration: requires a live Postgres (docker compose up -d)
```

- **Unused imports** (`ruff --select F401`), harmless but worth a pass:
  - `typing.Optional` — `agents/compliance_agent.py`, `agents/honeypot_agent.py`, `eval/active_learning.py`, `federated.py`
  - `dataclasses.field` — `eval/active_learning.py`
  - `dataclasses.asdict` — `sse_worker.py`
  - unused function imports `run_self_play`, `match_rules` — `sse_worker.py`
  - unused `seed_dataset` import — `eval/active_learning.py`

CLEAN (verified, no action needed)

- No bare `except:`, no `eval`/`exec`/`shell=True`, no mutable default args, no `pickle`.
- SQL is fully parameterized (`%s` placeholders) once issue #1's connection-API bug is fixed — no injection risk found.
- 228/230 tests green; the 2 failures are expected (no live Postgres in this environment), not test-code issues.